Conflict-Averse Gradient Descent for Multi-task Learning

2021 NIPS

Bo Liu et al.

Presenter
이재윤

Fundamental Team
김채현, 이근배

Contents

1. Multi-Task Learning

Multi-Task Learning

- Multi-Task Learning은 여러 Task를 수행하는 모델을 학습시키는 방법론

- Encoder는 공유함으로써 Representation은 공유, Head는 분리함으로써 각 Task 수행
  - Ex) [Semantic Segmentation / Depth Estimation] or [Keypoint Detection / Edge Detection]

- Why Multi-Task Learning?
  ① 더 좋은 성능
  ② 학습과정의 효율성
  ③ 가벼운 모델

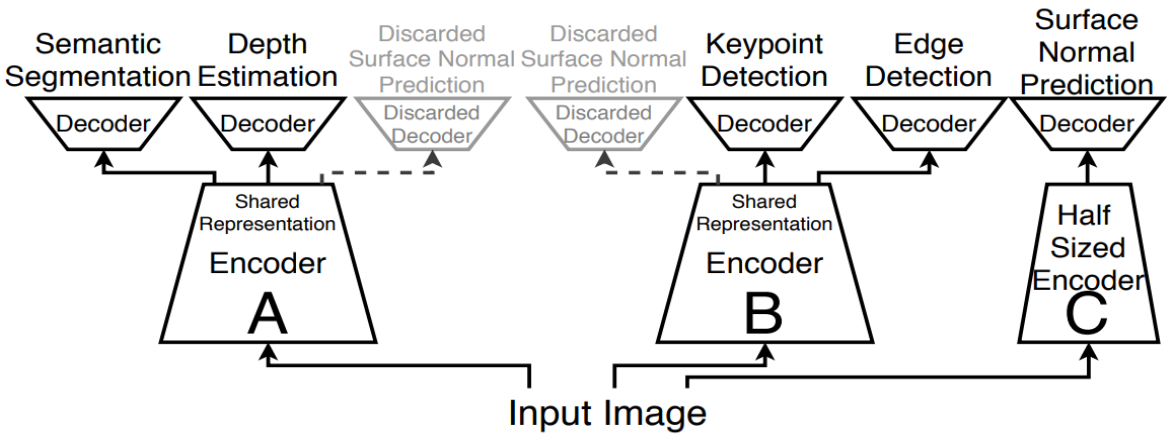


Fig 1. Multi-Task Learning Example[1]

[1] Which Tasks Should Be Learned Together in Multi-task Learning? (2020 ICLR)

Prior Works

1. Architectural approach
    - How to configure the architecture to perform multi-task learning
    - Hard Parameter Sharing / Soft Parameter Sharing

2. Task Relationship
    - **Negative Transfer** worsen the performance
    - Perform Multi-Task Learning considering Relevance between Tasks

3. Loss Weight
    - Learning speed and Loss Landscape of each task is different
    - Loss weights and/or gradient adjustment

Question

2. CAGrad

Problem Definition

- Given $K \geq 2$ tasks, **find optimal parameter $\boldsymbol{\theta}^* \in \mathbb{R}^m$**

$$\theta^* = \arg\min_{\theta \in \mathbb{R}^m} \left\{ L_0(\theta) \triangleq \frac{1}{K} \sum_{i=1}^{K} L_i(\theta) \right\}$$

- Optimization Challenge : ***Conflicting Gradients***
  - ✓ Denote $g_i = \nabla L_i(\theta)$, the gradient of task i and

    $g_0 = \frac{1}{K} \sum_i g_i$, the averaged gradient
  - ✓ g_0 may conflict with individual gradients,i.e. ∃i, $\langle g_i, g_0 \rangle < 0$
  - ✓ Then, certain task's performance may **sacrifice**

Conflict-Averse Gradient

- Simple Idea

"Looks for an update vector that maximizes the worst local improvement of any objective in a neighborhood of the average gradient"

- Assume we update $\theta' \leftarrow \theta - \alpha d$, where α is a step size and d an update vector

- Choose d to decrease not only the average Loss L_0, but also every individual loss

- Consider the minimum decrease rate across the losses

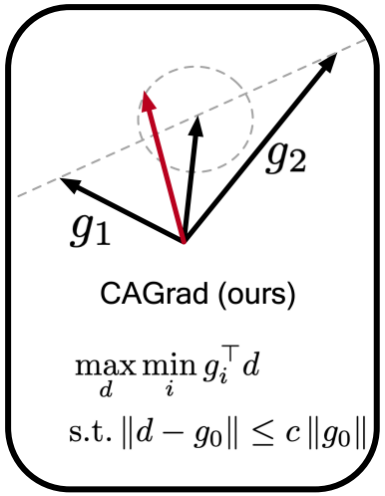
$$R(\theta, d) = \max_{i \in [K]} \left\{ \frac{1}{\alpha} (L_i(\theta - \alpha d) - L_i(\theta) \right\}$$

$$\cong \max_{i \in [K]} \frac{1}{\alpha} \{ L_i(\theta) - g_i \alpha d - L_i(\theta) \} = - \min_{i \in [K]} \langle g_i, d \rangle$$

using first-order Taylor approximation assuming small α.

Conflict-Averse Gradient

- If $R(\theta, d) = -\min_{i \in [K]} \langle g_i, d \rangle < 0$, all losses are decreased.
- Therefore, $R(\theta, d)$ can be regarded as the **measurement** of conflict among tasks



- Optimization problem of this paper can be presented as below:

$$\max_{d \in \mathbb{R}^m} \min_{i \in [K]} \langle g_i, d \rangle \text{ s.t } \|d - g_0\| \leq c \|g_0\|$$

- c is a pre-specified hyper-parameter
- g_0 can be the gradient of some other user-specified objective

Dual Objective

- Solving the optimization problem for d that has the same dimension as the number of θ is not practical

- The dual problem of the original optimization problem only involves solving for variable $w \in \mathbb{R}^K$ is more efficient

  Note, $\min_i \langle g_i, d \rangle = \min_{w \in W} \langle \sum_i w_i g_i, d \rangle$, where $\mathrm{w} = (w_1, \ldots, w_K) \in \mathbb{R}^K$

  and W denotes probability simplex $W = \{w: \sum_i w_i = 1 \; and \; w_i \geq 0\}$

- Using Lagrangian of the objective, optimization problem can be views as below:

$$\max_{d \in \mathbb{R}^m} \min_{i \in [K]} \langle g_i, d \rangle \;\; \textbf{s.t} \;\; \|d - g_0\| \leq c \, \|g_0\|$$

$$=$$

$$\max_{d \in \mathbb{R}^m} \min_{\lambda \geq 0, i \in [K]} g_w^{\mathrm{T}} d - \lambda(\|g_0 - d\|^2 - \phi)/2$$

  where $g_w = \sum_i w_i g_i$, and $\phi = c^2 \|g_0\|^2$

- By Slaster's condition, min / max can be switched.

Algorithm

- If we find a value equal to 0 by **partial differentiation** of the expression below with respect to w and d,

$$\max_{d\in\mathbb{R}^m} \min_{\lambda\geq 0, i\in[K]} g_w^{\mathrm{T}} d - \lambda(\|g_0 - d\|^2 - \phi)/2$$

- Optimal values can be found as below:

$$d = g_0 + g_w/\lambda$$
$$\lambda = \|g_w\|/\phi^{1/2}$$

- Then, we end up with the following optimization problem w.r.t w

$$w^* = \arg\min_{w\in W} g_w^{\mathrm{T}} g_0 + \sqrt{\phi}\|g_w\|$$

Algorithm

- Then, the parameter can be updated as below:
  - Initial model parameter θ_0, differential loss functions $\{L_i\}_{i=1}^{K}$, a constant $c \in [0,1)$ and learning rate α

Repeat

At the $t-th$ optimization step, define $g_0 = \frac{1}{K}\sum_i g_i$ and $\phi = c^2\|g_0\|^2$

Solve

$$min_{w \in W} F(w) \coloneqq g_w^{\intercal} g_0 + \sqrt{\phi}\|g_w\|$$

Update

$$\theta_t = \theta_{t-1} - \alpha(g_0 + \frac{\phi^{\frac{1}{2}}}{\|g_w\|} g_w)$$

Until convergence

Pareto Concepts

- *Pareto Dominated*

  For two points $\theta, \theta' \in \mathbb{R}^m$, θ is dominated by θ', denoted by $\boldsymbol{L}(\boldsymbol{\theta}') < \boldsymbol{L}(\boldsymbol{\theta})$,

  if $L_i(\theta') < L_i(\theta)$ for all $i \in [K]$ and $L_i(\theta') \neq L_i(\theta)$

- *Pareto-optimal*

  θ is *Pareto-optimal* if there exists no $\theta' \in \mathbb{R}^m$ such that $\boldsymbol{L}(\boldsymbol{\theta}') < \boldsymbol{L}(\boldsymbol{\theta})$.

  The set of all *Pareto-optimal* points is called **Pareto set**

- *Pareto-stationary*

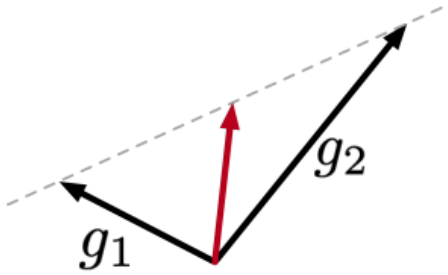
  θ is Pareto-stationary if we have $\min_{w \in W} \|g_w(\theta)\| = 0$,

  where $g_w(\theta) = \sum_{i=1}^{K} w_i \nabla L_i(\theta)$ and W is the probability simplex on [K]

  - In case of singe-objective, *Pareto optimal* point is *Pareto stationary*

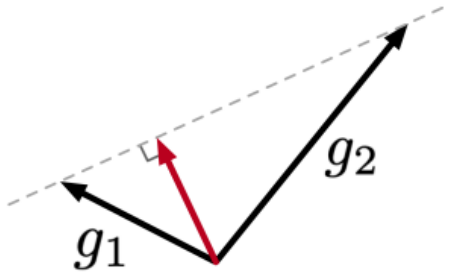
Remark

- When $c = 0$, $d = g_0$. CAGrad is equal to Gradient Descent
- When $c \to \infty$, CAGrad is equal to MGDA which minimize only $\|g_W\|$.
  - ✓ $min_{w \in W} F(w) \coloneqq g_w^{\intercal} g_0 + \sqrt{\boldsymbol{\phi}} \|g_w\|$
  - ✓ Weight 1 is given to the minimum size of the gradient, 0 to the rest of the gradients.
  - ✓ Converges to Pareto set
- When $0 \leq c \leq 1$, CAGrad converges to optimal point.



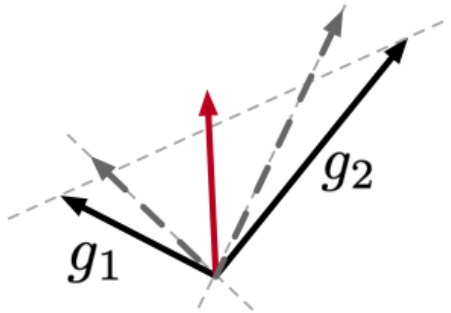
GD

$$d = (g_1 + g_2)/2$$



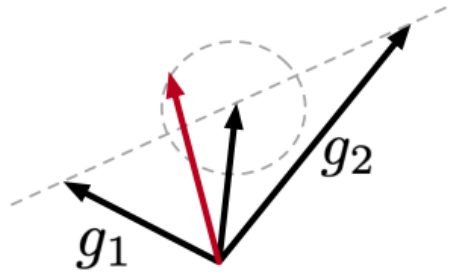
MGDA

$$\max_d \min_i g_i^{\top} d$$
$$\text{s.t. } \|d\| \leq 1$$



PCGrad

$$d = (g_{1 \perp 2} + g_{2 \perp 1})/2$$
$$\text{where } g_{i \perp j} = g_i - \frac{g_i^{\top} g_j}{\|g_j\|} g_j$$



CAGrad (ours)

$$\max_d \min_i g_i^{\top} d$$
$$\text{s.t. } \|d - g_0\| \leq c \|g_0\|$$

Prior Methods

- Existing methods have several problems
  - ✓ Gradient Descent with Adam gets stuck on valley. Initial point plays important role
  - ✓ MGDA / PCGrad converges to Pareto Set which is not the global optimization point
    - These methods over-optimize on objective while overlooking others
  - ✓ CAGrad(Proposed Method) converges to optimization point

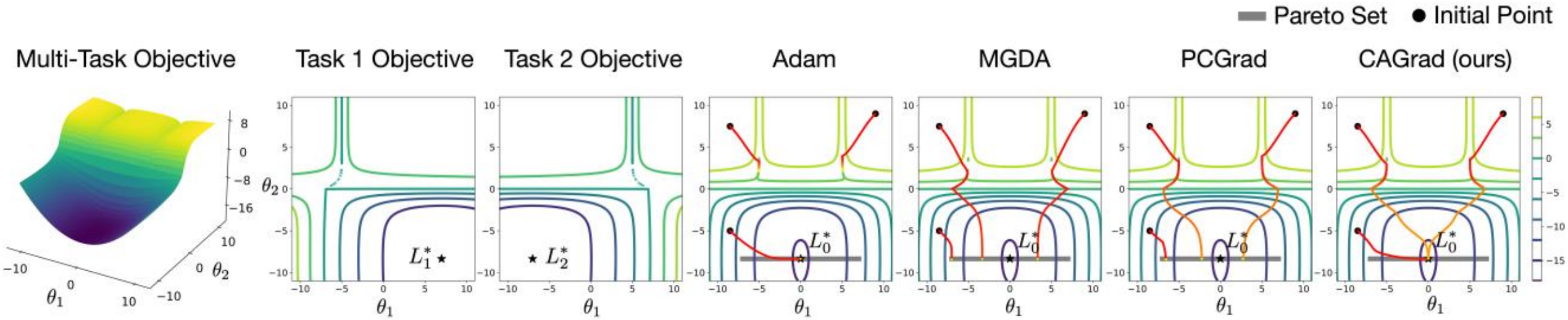


Fig 2. Toy Experiments

Convergence of CAGrad

- When $c < 1$, CAGrad is guaranteed to converge to a minimum point of the average loss L_0

- For all losses, let's assume their gradients are all H-Lipschitz(gradient does not diverge)
  1) For any $c \geq 1$, all the fixed points of CAGrad are Pareto-stationary points of $(L_0, L_1, \dots, L_K)$
  2) In particular, if we take $0 \leq c < 1$, then CAGrad satisfies

$$\sum_{t=0}^{T} \|\nabla L_0(\theta_t)\|^2 \leq \frac{2(L_0(\theta_0) - L_0^*)}{\alpha(1 - c^2)}$$

- When $0 \leq c < 1$, the sum of gradients have infimum value leading to the stationary point where $\nabla L_0(\theta_\infty) = 0$

- Unlike MGDA which stops when reached to the pareto set, CAGrad converges to the pareto stationary point

Question

3. Experiments

Experiments settings

- Questions
  1) Do CAGrad, MGDA, and PCGrad behave consistently with their theoretical properties in practice?
  2) Does CAGrad recover GD and MGDA when varying the constant c?
  3) How does CAGrad perform in both performance and computational efficiency?

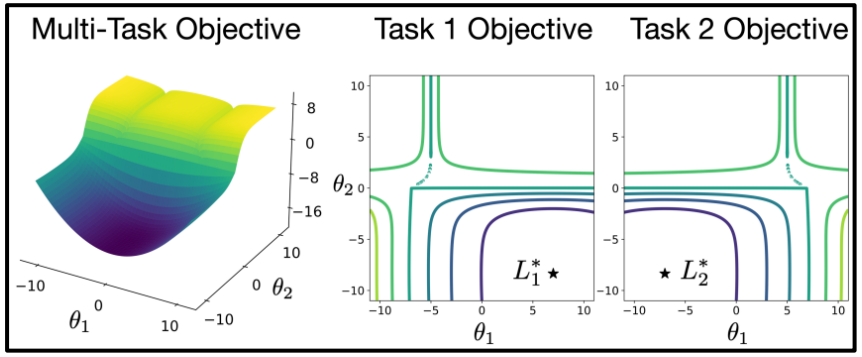
- Experiments
  I. Toy experiments & FashionMNIST
     - ✓ Answers for Q1 and Q2

  II. Benchmark
     - ✓ Answers for Q3

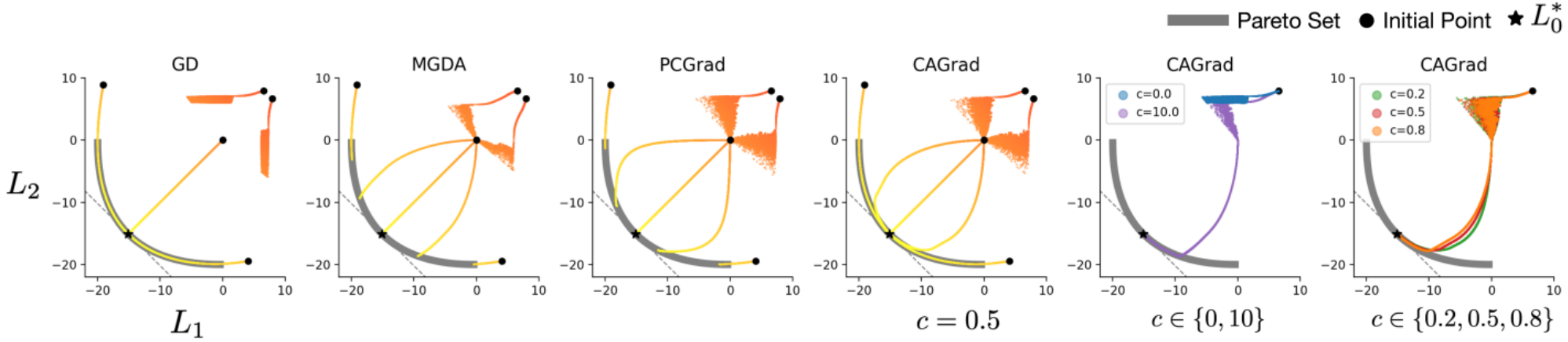
Toy Experiments

- Consider following individual loss functions:

$$
\begin{aligned}
&L_1(\theta) = c_1(\theta) f_1(\theta) + c_2(\theta) g_1(\theta) \text{ and } L_2(\theta) = c_1(\theta) f_2(\theta) + c_2(\theta) g_2(\theta), \text{ where} \\
&f_1(\theta) = \log\left(\max(|0.5(-\theta_1 - 7) - \tanh(-\theta_2)|,\ 0.000005)\right) + 6, \\
&f_2(\theta) = \log\left(\max(|0.5(-\theta_1 + 3) - \tanh(-\theta_2) + 2|,\ 0.000005)\right) + 6, \\
&g_1(\theta) = \left((-\theta_1 + 7)^2 + 0.1 * (-\theta_2 - 8)^2\right)/10 - 20, \\
&g_2(\theta) = \left((-\theta_1 - 7)^2 + 0.1 * (-\theta_2 - 8)^2\right)/10 - 20, \\
&c_1(\theta) = \max(\tanh(0.5 * \theta_2),\ 0) \text{ and } c_2(\theta) = \max(\tanh(-0.5 * \theta_2),\ 0).
\end{aligned}
$$



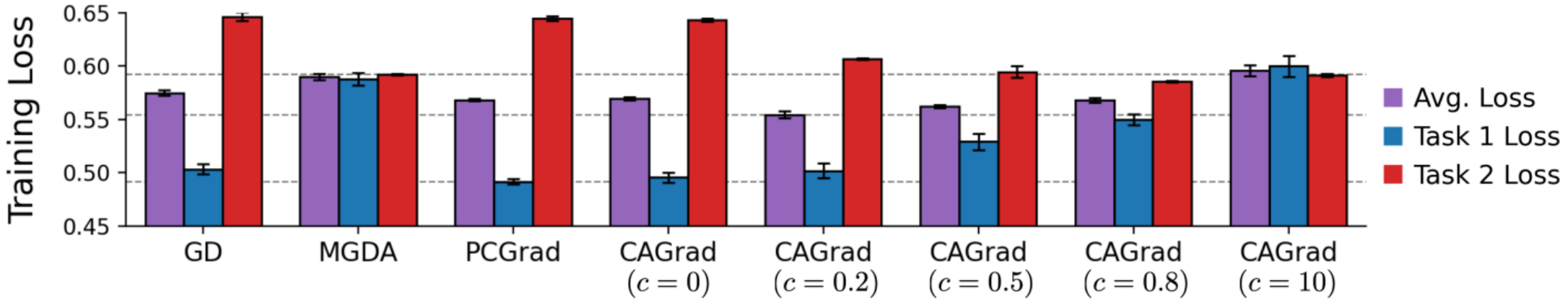
- Remark
  - ✓ GD get stuck in 2 out of 5 runs while others all converge to the *Pareto set*
  - ✓ MGDA and PCGrad converge to different Pareto-stationary depending on θ_{init}
  - ✓ CAGrad with c=0 recovers GD, CAGrad with c=10 recovers MGDA



Multi-Fashion + MNIST

- Benchmark consists of images generated by overlaying an image of FashionMNIST on top of an image from MNIST
- Use shrinked LeNet / Adam optimizer / 0.001 learning rate / 50 epochs / 256 batch size

- Remark
  - ✓ CAGrad achieves the lowest average loss with c=0.2
  - ✓ CAGrad with c=0 recovers GD, CAGrad with c=10 recovers MGDA
    - → As c increases, CAGrad acts more like MGDA



Multi-task Supervised Learning

- Benchmarks

  1) NYU-v2

     - Contains 3 tasks ∋ {Segmentation , Depth , Surface Normal}
     - CAGrad *combined with MTAN(SOTA MTL method)* **compares** {MTAN , PCGrad , Cross-stitch}
     - All tasks weights equally for fairness

  2) CityScapes

     - Contains 2 tasks ∋ {Segmentation, Depth}

#P.	Method	Segmentation (Higher Better)		Depth (Lower Better)		Surface Normal: Angle Distance (Lower Better)		Surface Normal: Within t° (Higher Better)			$\Delta m\% \downarrow$
		mIoU	Pix Acc	Abs Err	Rel Err	Mean	Median	11.25	22.5	30	
3	Independent	38.30	63.76	0.6754	0.2780	25.01	19.21	30.14	57.20	69.15	
≈3	Cross-Stitch [25]	37.42	63.51	0.5487	**0.2188**	*28.85	*24.52	*22.75	*46.58	*59.56	6.96
1.77	MTAN [21]	39.29	65.33	0.5493	0.2263	*28.15	*23.96	*22.09	*47.50	*61.08	5.59
1.77	MGDA [30]	*30.47	*59.90	*0.6070	*0.2555	**24.88**	**19.45**	**29.18**	**56.88**	**69.36**	1.38
1.77	PCGrad [41]	38.06	64.64	0.5550	0.2325	*27.41	*22.80	*23.86	*49.83	*63.14	3.97
1.77	GradDrop [4]	39.39	65.12	**0.5455**	0.2279	*27.48	*22.96	*23.38	*49.44	*62.87	3.58
1.77	CAGrad (ours)	**39.79**	**65.49**	0.5486	0.2250	26.31	21.58	25.61	52.36	65.58	**0.20**

#P.	Method	Segmentation (Higher Better)		Depth (Lower Better)		$\Delta m\% \downarrow$
		mIoU	Pix Acc	Abs Err	Rel Err	
2	Independent	74.01	93.16	0.0125	27.77	
≈3	Cross-Stitch [25]	*73.08	*92.79	*0.0165	*118.5	90.02
1.77	MTAN [21]	75.18	93.49	*0.0155	*46.77	22.60
1.77	MGDA [30]	*68.84	*91.54	0.0309	**33.50**	44.14
1.77	PCGrad [41]	75.13	93.48	0.0154	42.07	18.29
1.77	GradDrop [4]	**75.27**	**93.53**	*0.0157	*47.54	23.73
1.77	CAGrad (ours)	75.16	93.48	**0.0141**	37.60	**11.64**

4. Conclusion

Summary

- Contributions
  I. Introduce Conflict-Averse Gradient descent (CAGrad)
  II. Show convergence of the method
  III. Demonstrates improved performance

- Limitations
  I. CAGrad focus on optimizing the average loss
  II. Individual performance slightly degraded

Thank you